

Bloco 4.8, Propostas: seed e roteiro de teste nos 4 perfis — status de entrega

Data: 2026-09-08. Branch: feature-dev-breno-automacao. Working tree, sem commit.

O que o bloco é

Fechamento do sprint: dados de teste em gc-dev e verificação de permissão nos quatro perfis que tocam Propostas. É o bloco que responde "isso funciona pra quem não é admin?" — pergunta que nenhuma tela responde sozinha.

Atividade 1: supabase/seed_propostas.sql com tag [SEED-TESTE]

Concluída. 201 linhas, no padrão do `seed_orcamentos.sql` : bloco de insert, query de verificação e bloco de cleanup comentado no fim.

Uma diferença estrutural obrigou a mudar a forma: **proposta liga em obra, não em cliente**. O `seed_orcamentos.sql` cria os 28 clientes de que precisa e casa por nome; propostas dependem de obra já existente, com FK composta `(obra_id, empresa_id)`, e o cliente chega por obra.

Duas decisões daí:

- 1. **Rodízio entre as obras existentes**, com `row_number() over (...)` e módulo, em vez de casar por `codigo_obra`. Código de obra muda de ambiente pra ambiente; um seed que depende de `OBRA-2026-001` existir só funciona na máquina de quem escreveu. Assim roda com 1 obra ou com 20.
- 2. **Idempotente por `on conflict (empresa_id, numero) do nothing`**, aproveitando o unique que já existe. Rodar duas vezes não duplica nem estoura.

As 12 propostas cobrem de propósito o que as telas precisam mostrar:

CASO	POR QUE EXISTE
os quatro status	badge de cada cor, e o diálogo de status com destinos diferentes
uma enviada com validade vencida	o selo âmbar e o filtro <code>?vencidas=1</code> com dado positivo
duas sem validade	o "-" da coluna
<code>desconto = valor_total</code>	o CHECK <code>propostas_desconto_valido</code> é <code><=</code> , não <code><</code> — o <code>valor_final</code> sai 0,00
soma de <code>pct_*</code> fechando 100% e parcial (50%)	o CHECK <code>propostas_pct_soma</code> também é <code><=</code> : soma parcial é válida
rejeitada com motivo <code>outro</code> + detalhe	o caso estrito do CHECK <code>propostas_rejeitada_motivo</code>

CASO	POR QUE EXISTE
<code>historico</code> já preenchido, coerente com o status	a aba Histórico não abre vazia

O `por` do histórico fica `null` de propósito: seed não tem autor, e apontar para um profile que não fez a mudança seria registro falso. A aba mostra a data e omite o nome.

Rodado em `gc-dev`, com a query de verificação do próprio arquivo devolvendo as 12 linhas — `valor_final` calculado pelo banco (coluna `generated`), `pct_soma` conferindo, e as transições de histórico batendo com o status.

Atividades 2 a 5: o roteiro nos quatro perfis

Concluídas, e automatizadas em vez de checklist manual: entraram na camada `runtime` do plano de validação (`scripts/validacao-rotas.json`), que faz login com o usuário de cada perfil e afere a tela.

As senhas dos `@teste.com` de `gc-dev` eram desconhecidas (a do `setup_inicial_dev.sql` não confere mais). A primeira saída foi redefini-las com `scripts/resetar-senha-dev.mjs` ; **desde 2026-09-08 não é mais necessário** — a validação gera sessão sem senha via `scripts/sessao-dev.mjs` (ver pendência 3). O script de reset fica, porque é útil pra entrar num perfil de teste pela tela.

Admin: CRUD completo

Concluída. A camada `escrita` roda o ciclo inteiro contra `gc-dev` — criar, editar, mudar status (com histórico), anexar, remover anexo e excluir —, mais as recusar: número repetido, desconto maior que o total, editar proposta enviada, reabrir proposta terminal. 35 passos, com limpeza do que cria.

Comercial: tudo menos excluir

Concluída. Três rotas, autenticadas como `comercial@teste.com` :

ROTA	ASSERÇÃO
<code>/propostas</code>	vê a listagem e o botão "Nova proposta"
<code>/propostas/nova</code>	200 — abre o formulário, não redireciona
<code>/propostas/[id]</code>	vê "Mudar status" e não vê "Excluir"

O `naoEsperaHtml: ["Excluir"]` é o que prova a regra: exclusão é só admin, tanto no botão quanto na action (`deleteProposta` recusa por permissão, verificado na camada de escrita).

Visualizador: só leitura

Concluída. Quatro rotas: a listagem **sem** "Nova proposta", o detalhe **sem** "Mudar status" nem "Excluir", e `/propostas/nova` e `/propostas/[id]/editar` devolvendo 307.

O redirect é o ponto: o `layout.tsx` de propostas libera `visualizador` (ele precisa ler), então as rotas de escrita têm guard próprio na page. Sem isso, o visualizador chegaria ao formulário e só descobriria a falta de permissão ao salvar.

Produção e medição: mesmo tratamento do financeiro

Concluídas em 2026-09-08. Não estavam no card — que pede quatro perfis —, mas a pendência dizia que o comportamento esperado era "o mesmo do financeiro" e isso era expectativa, não medição. Agora é medição: três rotas cada, no mesmo formato (menu sem "Propostas", `/propostas` em 307, e `/execucao` em 200 como contraprova).

Financeiro: Propostas fora do menu, e a rota redireciona

Concluída. Três rotas, como `financeiro@teste.com`:

ROTA	ASSERÇÃO
<code>/</code>	o menu não traz "Propostas" (<code>nav.ts</code> não lista o perfil)
<code>/propostas</code>	307 — o layout não libera financeiro
<code>/fd</code>	200 — a área dele funciona

A terceira existe pra distinguir "sem acesso a Propostas" de "sem acesso a nada": sem ela, um erro de sessão passaria como sucesso do teste de permissão.

Atividade 6: build da Vercel verde e console limpo

Concluída, as duas metades.

Console limpo: a camada `navegador` coleta `Log.entryAdded` e `Runtime.exceptionThrown` durante o roteiro inteiro — login, formulário, diálogo, abas, exclusão — e reprova se houver erro. Passou com (nenhum), descontando ruído conhecido (favicon e o aviso do React DevTools).

Build da Vercel: o deploy Preview do commit `e74bc77`, que é o HEAD desta branch, está **success**:

```
Preview e74bc775 → success
https://gc-sistema-qei06cj61-cremalets-projects.vercel.app
```

Conferido pela API do GitHub (`repos/cremalet/gc-sistema/deployments`), não pelo painel. Não existe `vercel.json` nem CLI da Vercel no projeto — o deploy é disparado pela integração com o repositório, o que também explica por que só apareceu depois de a branch chegar ao `cremalet`.

Verificação

CAMADA	RESULTADO
estático	<code>tsc --noEmit</code> limpo, lint sem warnings
unitário	46 casos, <code>fail 0</code>
build	31 rotas, iguais à baseline
runtime	39/39 rotas , sendo 16 em perfil não-admin (4 visualizador, 3 comercial, 3 financeiro, 3 produção, 3 medição)
dados	12/12 checagens sob RLS em gc-dev
escrita	35/35 passos na medição de 2026-09-08 — número velho : três checagens de perfil pararam de rodar naquele mesmo dia, e hoje são 38/38. Ver <code>docs/tecnicos/auditoria-cobertura-sprint-4.md</code>
navegador	22/22 passos clicando na tela, console limpo
Vercel	Preview de <code>e74bc77</code> success

Pendências

Revisadas em 2026-09-08: as três próprias do bloco foram resolvidas.

- ~~1. `producao` e `medicao` seguem sem cobertura.~~ **Resolvida — agora é medição, não expectativa.** Seis rotas novas, três por perfil: o menu **sem** "Propostas", `/propostas` devolvendo `307`, e `/execucao` em `200` como contraprova de que a sessão existe. Os seis perfis do `setup_inicial_dev.sql` que tocam (ou não tocam) Propostas estão medidos: admin, comercial, visualizador, financeiro, produção e medição. São **39 rotas** na camada runtime, 16 delas em perfil não-admin.

O que destravou isso foi a pendência 3: sem senha, não havia como logar.

- ~~2. O seed não roda por script.~~ **Resolvida.** `scripts/aplicar-seed.sh` (`npm run seed`) aplica qualquer `supabase/seed_*.sql` em gc-dev por um comando, com `--verificar` para rodar só o bloco de verificação do arquivo. Tem a mesma trava de ambiente dos outros scripts.

Corrigindo a formulação anterior: esta pendência dizia "não roda no CI". **Não existe CI neste projeto** (`.github/workflows/` não existe), então "rodar no CI" era um requisito que eu inventei. O problema real era outro e mais simples: aplicar o seed dependia de abrir o navegador, achar o arquivo e colar no SQL Editor — não era repetível. Agora é. Os arquivos continuam escritos para o SQL Editor, que segue sendo o caminho documentado.

- ~~3. As senhas dos `@teste.com` estão em texto claro no `.env.local`.~~ **Resolvida, e eliminada em vez de escondida.** `scripts/sessao-dev.mjs` gera sessão pelo Admin API (`generateLink` + `verify0tp`), usando o service role que o `.env.local` já precisa ter. **Nenhuma senha de perfil de teste é guardada em lugar nenhum:** o `.env.local` saiu de oito variáveis de validação para duas (só a do admin, que loga pela tela de verdade).

Efeito colateral bom: perfil novo na validação virou **uma linha** em `PERFIS_DE_TESTE` , sem reset de senha nem variável nova. Foi assim que `producao` e `medicao` entraram. O helper só aceita `@teste.com` e só roda contra gc-dev — gerar sessão de conta real, mesmo em dev, é fabricar acesso ao usuário de alguém.

4. **Herdada — risco aceito em 2026-09-10, não é mais pendência.** Revogar os dois personal access tokens e trocar a senha do admin de gc-dev, vazados em repositório público, era o que restava do bloco 4.1. Decisão do dono da conta: não trocar. O efeito registrado está em `docs/4.1-status-entrega.md` .

Arquivos

ARQUIVO	O QUE É
<code>supabase/seed_propostas.sql</code>	novo. 12 propostas com tag <code>[SEED-TESTE]</code> , rodízio entre obras
<code>scripts/resetar-senha-dev.mjs</code>	reset de senha de <code>@teste.com</code> em gc-dev
<code>scripts/validacao-rotas.json</code>	32 rotas, 10 em perfil não-admin
<code>scripts/validar-runtime.mjs</code>	login por perfil (visualizador, comercial, financeiro)
<code>.env.local</code>	credenciais dos perfis de validação (gitignored)
<code>docs/4.8-status-entrega.md</code>	este documento